

Project #1
Signals, Pipes & Fifos under Linux
Due: May 3, 2026

Instructor: Dr. Ahmad Afaneh, Dr. Hanna Bullata

Home furnishing competition using signals, pipes & fifos

We would like to create a multi-processing application that simulates the behavior of 2 teams competing to furnish 2 houses using signals, pipes & fifos. Each team will do its best to finish the furnishing job of its house before the other team so as to win the competition. the home furnishing simulation can be described as follows:

1. The 2 teams have a user-defined number of members. The members are numbered starting 0 up to the user-defined number - 1. Member 0 of each team is the source member located next to its furniture pile while the member whose number is user-defined - 1 is the sink member located in the respective house of each team.

The team members are aligned side by side and each member will get a piece of furnisher from the previous team member and pass it to the next team member until that piece of furnisher reaches the sink member.

2. The furniture is composed of user-defined number of pieces (e.g. default is 1000 pieces) and each piece comes in a pair of 2 (1 for each house). The furniture pieces located at the source member are numbered serially. The number is assigned either randomly by the system or provided by the user. However, no 2 pieces of furniture are allowed to have the same assigned number.

Team members are unaware of the serial number that has been assigned to the furniture and thus do not know the order in which they should move the furniture.

3. The source member of each team will select a random piece of furniture and pass it to the next team member. That piece of furniture will move from team member to the next team member until it reaches the sink member.

If the piece of furniture reaches the sink member in the correct order, it is kept in the house. Otherwise, that piece of furniture will go back in the reverse order from team member to the previous team member until it reaches the source member. If that situation arises, the source member should not select that same piece of furniture again until a successful move of a different piece of furniture takes place. By successful, we mean a piece of furniture was moved to the team house in the correct order.

4. When moving furniture from source to sink, each team member should pause a random amount of time that is random but bound to a user-defined range. The pause time of team members should increase with time since team members become obviously tired from moving furniture around.
5. When any of the 2 teams succeed in moving all furniture pieces in the correct order to their house, that team is declared the winner of the round. The next round starts by placing back all furniture of the 2 houses at the location of the source team members and each furniture piece is assigned a new serial number. A new round starts by going back to step 3. above.
6. The competition ends when any of the 2 teams has won a user-defined number of times.

What you should do

- In order to implement the above-described application, you need to use the signals, pipes and/or fifos facilities to do communication and data sharing between processes (you do not need to use all of them). Be wise in the choices you make and be ready to convince us that you made the best choices :-)
- Write the code for the above-described application using a multi-processing approach. Use openMP wherever needed to further accelerate the operations by parallelizing loops. Check if you get better or worse results when using it.
- In order to avoid hard-coding values in the code, think of creating a text file that contains user-defined values or ranges and pass the file name as an argument to the application so that new values can be loaded with each run. That will spare you from having to change your code permanently and re-compile.
- Use graphics elements from opengl library in order to best illustrate the application. Nothing fancy, just simple and elegant elements are enough.
- Test your program.
- Check that your program is bug-free. Use the `gdb` debugger in case you are having problems during writing the code (and most probably you will :-). In such a case, compile your code using the `-g` option of the `gcc`.
- The project lead is responsible to send the zipped folder that contains your source code and your executable before the deadline on behalf of the team. If the deadline is reached and you are still having problems with your code, just send it as is!